Open in app ↗

Medium

Neural Notions · Follow publication

Member-only story

Featured

Andrej Karpathy Stopped Using AI to Write Code. He's Using It to Build a Second Brain Instead

12 min read · Apr 5, 2026

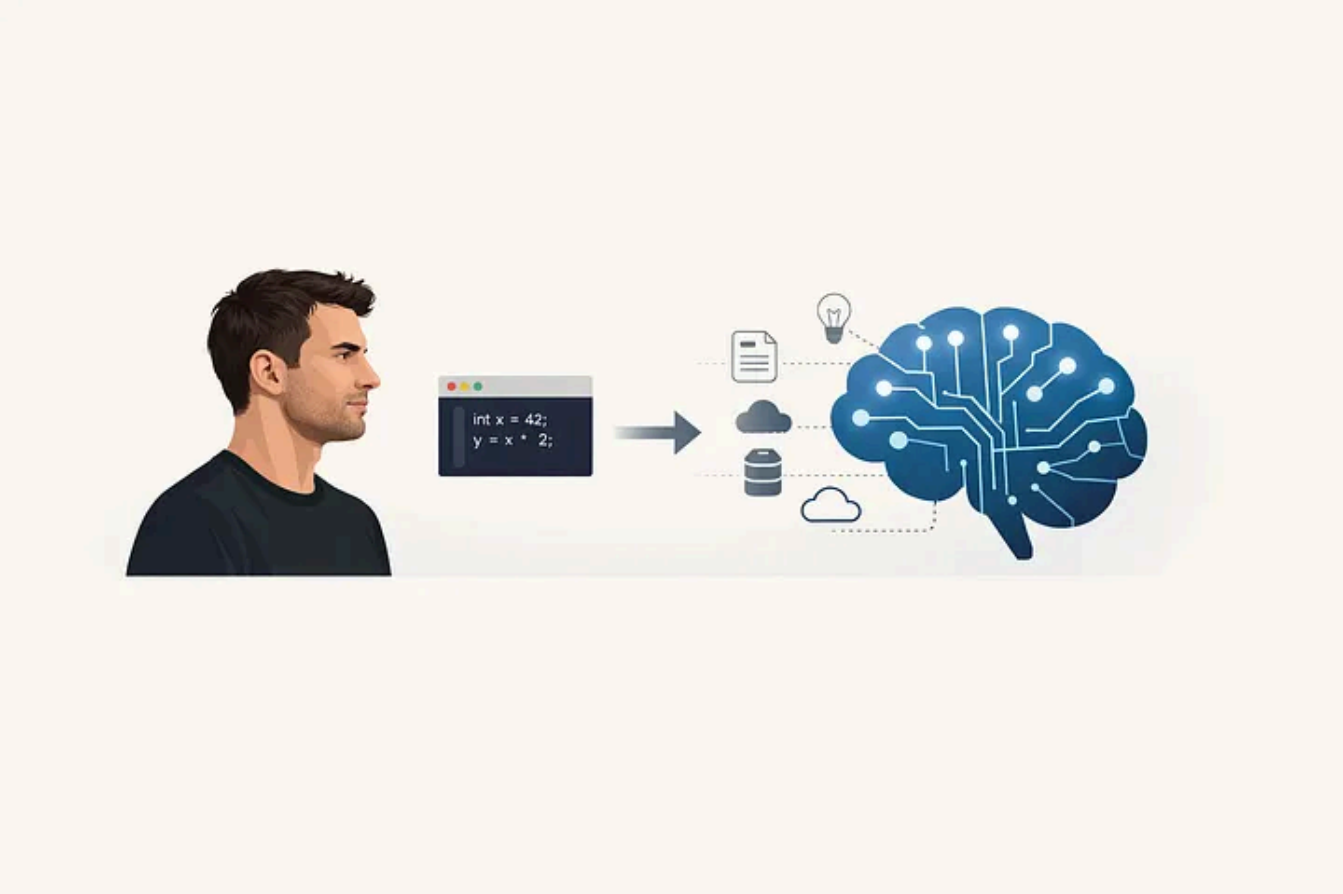
Nikhil

Follow

Listen

Share

More



Andrej Karpathy — Knowledge base

His new workflow turns raw research into a self-maintaining wiki.No vector databases, no RAG pipelines, just markdown files and an LLM that acts like a full-time librarian.

On April 3, 2026, **Andrej Karpathy** co-founder of *OpenAI*, former AI lead at Tesla, the guy who coined "**vibe coding**" posted something on X that stopped me mid-scroll.

He said he's spending less time using AI to generate code and more time using it to organize knowledge. Not in some abstract, theoretical way. He described an actual working system where he dumps raw research materials into a folder, points an LLM at it, and the LLM builds and maintains an entire interlinked wiki from scratch. No human editing. The AI writes the articles, creates backlinks between related ideas, categorizes concepts, and keeps the whole thing updated as new material comes in.

His research wiki on a single topic has grown to about 100 articles and 400,000 words. He rarely touches it directly.

The tweet went massively viral. The next day, he followed up with what he called an "**idea file**" a GitHub gist laying out the full architecture, designed to be copied and pasted directly into an LLM agent so it can build the system for you.

His reasoning: "*In this era of LLM agents, there is less of a point/need of sharing the specific code/app, you just share the idea, then the other person's agent customizes & builds it for your specific needs.*"

I want to break down exactly how this works, why it matters, and how you can build your own even if you've never set up a development environment in your life.

The problem this solves (and why RAG isn't the answer)

If you've used ChatGPT or Claude with uploaded files, you've used something called RAG, Retrieval-Augmented Generation.

Here's how it typically works: your documents get chopped into small chunks, those chunks get converted into mathematical representations called embeddings, and when you ask a question, the system searches for the most similar chunks and feeds them to the AI.

It works. But it has a fundamental limitation that becomes obvious once you use it for anything serious.

Every time you ask a question, the AI is rediscovering your knowledge from scratch. It searches, retrieves some chunks, and tries to piece together an answer. Nothing accumulates. Ask a question that requires connecting ideas from five different documents, and the system has to find and stitch together the right fragments every single time.

There's no memory. There's no structure. There's no map of how your ideas relate to each other.

Karpathy's approach is different. Instead of searching through raw documents on every query, the LLM reads the raw material once and compiles it into a structured, organized wiki. Summaries, concept articles, backlinks, comparisons, an index the whole thing. When you ask a question later, the AI doesn't need to search through a vector database. It can just read the relevant wiki pages, which already contain the synthesized, organized version of the information.

As **VentureBea**t put it in their coverage, the LLM in this setup isn't acting as a search engine. It's acting as a research librarian, one who actively authors and maintains a persistent record. And because everything is stored as plain markdown files you can read yourself, there's no black box. Every claim can be traced back to a specific file that you can open, read, and edit in any text editor.

The architecture

Karpathy's system has several distinct layers. I'm going to walk through each one using the diagram he shared, but in terms that don't require a computer science degree.

Layer 1: Raw sources (your research material)

This is the bottom layer, the foundation. You collect articles, research papers, images, data files, code repositories, whatever is relevant to the topic you're researching. All of it goes into a folder called *raw*.

The rule here is simple: This folder is the source of truth. The LLM can read from it, but never writes to it or modifies it. Your original materials stay untouched.

For web articles, Karpathy uses the Obsidian Web Clipper browser extension, which converts web pages into markdown files with one click. He also downloads related images locally so the LLM can reference them directly instead of relying on web URLs that might break.

You don't need fancy tools for this step. Saving PDFs to a folder, copy-pasting articles into text files, downloading research papers all of that works. The point is to get your raw material into one place.

Layer 2: The schema (instructions for the LLM)

This is the part most people miss, and it's what makes the system work consistently.

Before the LLM touches any data, you give it a configuration file Karpathy calls it ***CLAUDE.md*** if you're using Claude Code, or ***AGENTS.md*** for other tools. This file tells the LLM how the wiki should be structured, what page formats to use, what naming conventions to follow, and how the ingest process should work.

Think of it like an employee handbook for the AI. Without it, the LLM would organize things differently every time. With it, every compilation run follows the same structure, producing consistent results.

You don't need to write this from scratch. Karpathy's GitHub gist is this file you can literally copy it, paste it into your LLM agent, and it will know how to build the wiki.

Layer 3: The wiki (what the LLM builds)

This is the core of the system. The LLM reads your raw sources and "compiles" them into a structured collection of markdown files. Karpathy's wiki includes several types of pages:

Entity pages cover specific people, organizations, or projects mentioned in the research. If you're researching AI safety and three papers mention the same researcher, the LLM creates a page about that person with information pulled from all three sources.

Concept pages cover ideas, methods, and theories. If multiple papers discuss the same technique, the LLM creates a dedicated article explaining it, with references back to the original sources.

Summaries are per-source digests. Each raw document gets a summary page that captures the key points without you having to read the entire original.
Comparison pages put related ideas side by side. If two papers propose competing approaches to the same problem, the LLM writes a comparison that draws out the differences.

Synthesis pages provide overviews that tie multiple sources together around a theme.

The wiki also includes two special files: index.md, which is a master catalog organized by category, and log.md, which keeps a chronological record of everything that's been added or changed.

Pages are interlinked with wiki-style [[backlinks]], use YAML frontmatter (metadata headers) for categorization, and can be tracked with Git for version history.

The critical thing to understand is that the LLM writes and maintains all of this. You don't manually create pages, write summaries, or add backlinks. The AI does that work. Your job is to curate what goes into the raw folder and to ask questions.

The operations: Ingest, Query, and Lint

Once the structure exists, three operations keep the system running:

Ingest is what happens when you add new material. You drop a new article or paper into the *raw/* folder. The LLM reads it, extracts the important information, and updates 10–15 wiki pages creating new concept pages if needed, updating existing ones, adding backlinks, refreshing summaries.

This is incremental. The LLM doesn't rebuild the entire wiki from scratch every time. It reads the new source and integrates it into the existing structure. Karpathy describes this as "**compiling**" a term borrowed from programming, where source code gets translated into something executable. Here, raw research gets translated into organized knowledge.

Query is what happens when you ask a question. The LLM searches the wiki's index, reads the relevant pages, and synthesizes an answer. Because the wiki already contains organized, interlinked summaries, the AI can answer complex questions that would require connecting information from many different sources something that traditional RAG systems struggle with.

At 400,000 words, Karpathy says the LLM navigates the wiki just fine using its own index files and summaries. No vector database needed. No embeddings. No similarity search. Just structured markdown that the AI can read and reason about.

Lint is the maintenance pass. Periodically, you tell the LLM to scan the wiki for problems: contradictions between pages, orphan pages that aren't linked from

anywhere, gaps where important topics are mentioned but don't have their own article, stale claims that might need updating. The LLM flags these issues and can fix many of them automatically.

As one community member put it in the responses to Karpathy's post: "It acts as a living AI knowledge base that actually heals itself."

The output layer

Here's where it gets practical. Instead of getting answers as plain text in a chat window, Karpathy has the LLM generate its outputs as files he can view in Obsidian:

Markdown pages that become part of the wiki itself. When he asks a complex question and gets a thorough answer, he "files" that answer back into the wiki. This means his own explorations and queries always compound every question makes the knowledge base richer.

Slide presentations in Marp format (a tool that converts markdown into slide decks). If he needs to present findings, the LLM generates slides directly.

Charts and visualizations using matplotlib (a charting library). Data from the wiki gets turned into visual representations.
Comparison tables that put ideas side by side in a structured format.

The key insight: the outputs feed back into the wiki. The system gets smarter every time you use it.

How to actually build this

You don't need to be a developer. Here's the minimum viable version.

Step 1: Install Obsidian. Go to obsidian.md and download the free app. It works on Mac, Windows, Linux, and mobile. Obsidian is a note-taking app that works with plain markdown files stored on your computer. It's your "IDE" the place where you'll view and browse the wiki.

Step 2: Create your folder structure. Inside your Obsidian vault (that's just the folder Obsidian uses), create two subfolders: *raw/* for your source materials and *wiki/* for the compiled output. That's it for the structure.

Step 3: Install the Obsidian Web Clipper. This is a free browser extension that lets you save any web page as a markdown file with one click. When you find an article

worth adding to your knowledge base, clip it into your raw/ folder.

Step 4: Start collecting raw materials. Pick a topic you're researching or learning about. Save 5–10 articles, papers, or other documents into raw/. Don't worry about organization at this stage that's the LLM's job.

Step 5: Set up your LLM agent. This is the part that requires the most technical comfort, but it's gotten dramatically easier in 2026. You have several options:

If you use Claude, you can do this through Claude's file upload feature for small wikis. For larger ones, Claude Code (Anthropic's command-line tool) can work directly with files on your computer.

If you use Codex, Cursor, or another coding agent, they can also read and write files on your local machine.

The simplest approach for non-technical users: open your preferred AI chat tool, upload the contents of your ***raw/*** folder, paste Karpathy's idea file (from his GitHub gist) as instructions, and ask the LLM to compile a wiki from your sources. Save the output markdown files into your ***wiki/*** folder.

Step 6: Copy Karpathy's idea file. Go to *gist.github.com/karpathy/442a6bf555914893e9891c11519de94f*. Copy the entire gist. This is your schema the instructions that tell the LLM how to build and maintain the wiki. Paste it as context when you start your compilation session.

Step 7: Run the first compilation. Tell the LLM something like: "Read all the documents in the ***raw/*** folder. For each one, write a summary. Then identify the major concepts, people, and organizations across all sources. Create wiki pages for each, with backlinks. Create an index.md that catalogs everything by category, and a log.md that records what was processed."

The LLM will generate a set of markdown files. Save them into your ***wiki/*** folder. Open Obsidian and browse.

Step 8: Start querying. Now you can ask questions against your wiki. Give the LLM access to the wiki files and ask: "B*ased on the wiki, what are the main disagreements between authors X and Y on topic Z?*" or "Summarize everything we know about concept A and how it relates to concept B." The answers will be grounded in your actual research, not the LLM's general training data.

Step 9: File outputs back. When you get a good answer, save it as a new markdown file in the wiki. Over time, your explorations and the wiki's compiled knowledge merge into something richer than either one alone.

Step 10: Run lint passes. Every few weeks (or whenever the wiki grows substantially), ask the LLM to scan for problems: orphan pages, contradictions, missing concept articles, stale information. Fix what it finds. The wiki gets tighter over time.

Why this is better than what you're probably doing now

Most people manage research in one of three ways: browser bookmarks they never revisit, a notes app with disconnected entries, or a chat history that disappears when the session ends. All three have the same problem: nothing connects to anything else, and nothing compounds over time.

Karpathy's system solves this because the LLM maintains the connections. When you add a new source that mentions a concept already covered in the wiki, the LLM updates the existing concept page and adds backlinks. You don't have to remember that you read about that concept three weeks ago in a different paper. The system remembers for you.

The other advantage is the separation between raw truth and compiled knowledge. Your original sources never get modified. The wiki is a layer on top — an interpretation that you can regenerate, edit, or throw away without losing any source material. If the LLM misunderstands something, you fix it in the wiki without touching the original.

And because everything is plain markdown on your own computer, there's no vendor lock-in. No subscription. No cloud dependency. If Obsidian disappears tomorrow, you still have a folder full of text files you can open in anything.

Where this gets really interesting

Karpathy mentions one future direction that's worth thinking about: once your wiki is clean, comprehensive, and well-linked, you could use it to generate synthetic training data and fine-tune a smaller LLM so it actually "*knows*" the information in its weights. You'd go from a knowledge base that the AI reads at query time to a specialized model that has internalized your entire research domain.

That's still an advanced use case. But the more immediate applications are already exciting. Entrepreneur Vamshi Reddy responded to Karpathy's post with an observation that stuck with me: "E*very business has a raw/ directory. Nobody's ever compiled it. That's the product.*"

Think about that. Every company has a mess of Slack threads, meeting transcripts, internal documents, product specs, and research that nobody can find or connect. The same wiki pattern works for all of it. Competitive analysis. Due diligence. Trip planning. Tracking characters and plot threads while reading a novel series. Learning a new technical field. Studying for an exam.

The system is domain-agnostic because the schema layer absorbs all the domain-specific configuration. Change the instructions in your CLAUDE.md file, and the same architecture compiles a different kind of wiki.

What it can't do

One thoughtful critique came from a writer on Substack who made a distinction worth keeping in mind. The LLM is excellent at the reconnaissance phase mapping the territory, organizing information, finding connections you might have missed. But synthesis actually forming original ideas from the material is still a human job.

The writer referenced the German sociologist Niklas Luhmann, who maintained a famous system of handwritten note cards. Luhmann's insight was that the friction of writing something in your own words isn't wasted effort. It's the actual mechanism through which understanding happens. Reading someone else's summary is not the same as formulating the idea yourself.

So the honest version of what Karpathy's system does: it eliminates the drudgery of organizing, connecting, and maintaining research materials work that humans are bad at and find tedious. It doesn't eliminate the need to think. The AI compiles the territory. You still have to walk it.

That's a good trade.

Getting started today

If you want to try this, here's the minimum you need:

1. Download Obsidian (free) from obsidian.md
2. Install the Obsidian Web Clipper browser extension

3. Create a vault with *raw/* and *wiki/* folders
4. Clip 5–10 articles on a topic you care about into *raw/*
5. Grab Karpathy's idea file from his GitHub gist
6. Paste it into your LLM of choice along with your raw files
7. Ask it to compile a wiki

Start small. You can always add more sources later. The system is designed to grow incrementally each new source gets integrated into the existing structure, not processed from scratch.

And if you find yourself, a few weeks later, with a 50-page interlinked wiki that you barely had to edit, a system that answers complex research questions by citing your own collected sources, and a knowledge base that gets better every time you use it well, then you'll understand why Karpathy's tweet went viral.

The bottleneck was never writing code. It was understanding the problem deeply enough to know what to build. Now there's a system for that.

Found this useful? Follow for more breakdowns of tools and workflows that actually change how you work not just how your demo looks.

AI | Artificial Intelligence | LLM | Large Language Models | Technology

Follow

Published in Neural Notions

195 followers · Last published 19 hours ago

Exploring Artificial Intelligence, Large Language Models, and the ideas shaping the future of intelligent systems. Follow and subscribe for AI and Technology trends

Follow

Written by Nikhil

416 followers · 13 following

Human with a story to tell, a story to preserve and a story still unfolding.

Responses (18)

Christian Broberg

What are your thoughts?

Dhruv
Apr 10

This hit home. Using AI as a thinking partner and knowledge organizer feels more durable than using it just to spit out code. The real advantage might be building systems that compound your understanding over time.

9 Reply

Sbayer
5 days ago

I think it should be named The Andrej Foundation, or just the Foundation. More proof that Asimov was a time independent entity

6 1 reply Reply

Cynthia Afogu
6 days ago

When the guy who built Tesla's Autopilot says he's stopped using AI for 'raw code' and started using it for 'knowledge organization,' the market needs to listen.
Most companies are still trying to force AI to 'do tasks.' At Vanguard Logic, we follow... more

7 Reply

See all responses

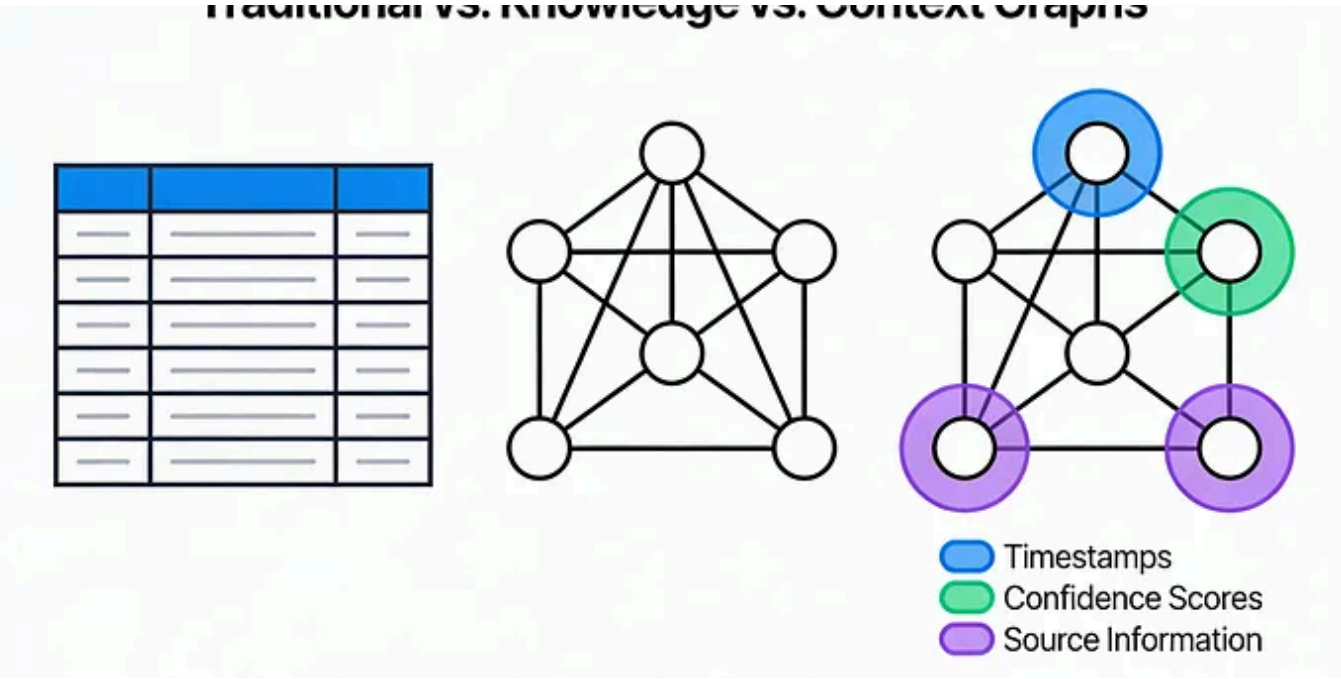
More from Nikhil and Neural Notions

In Neural Notions by Nikhil

Paperclip AI: Open source platform focused on turning ai agents into a company

Why the hardest problem in AI right now isn't making smarter agents, it's getting them to work together without burning through your API...

Apr 2 188 3

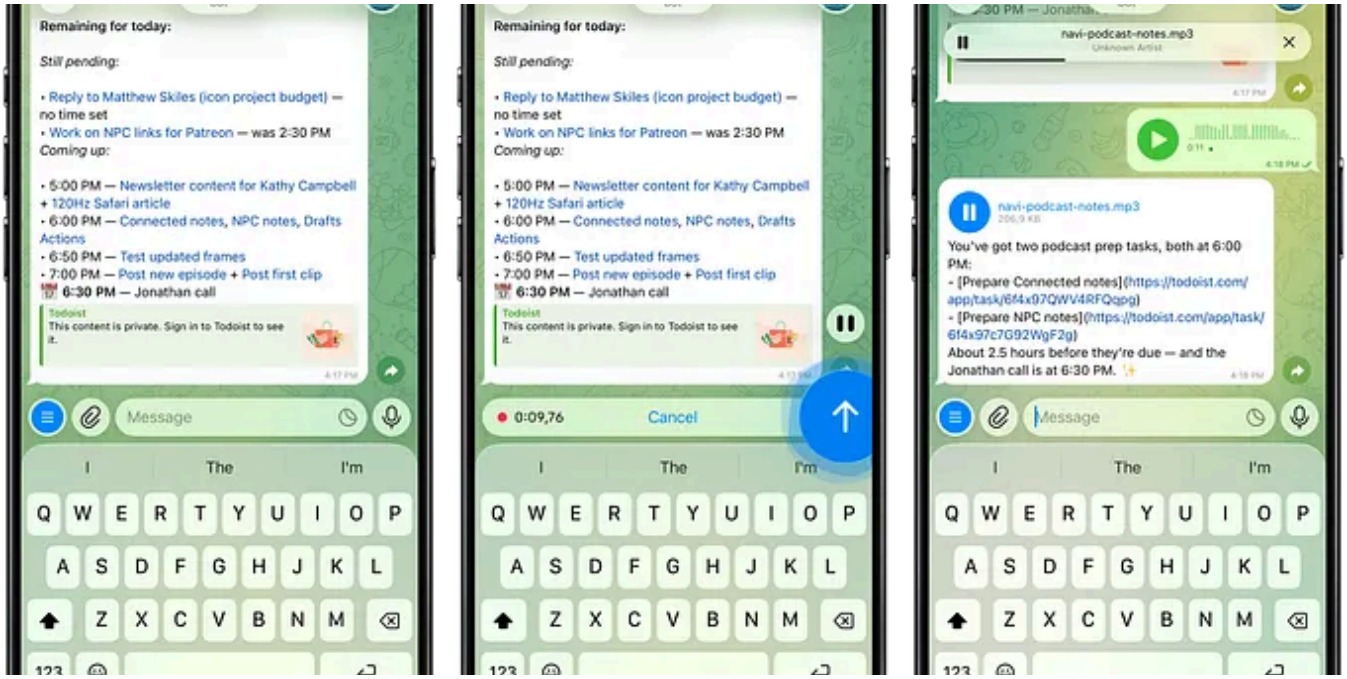


In Neural Notions by Nikhil

What are Context Graphs: Building the AI that trulyUnderstands

Imagine an AI system that not only processes information but truly understands it -> grasping the relationships between entities, the...

Dec 25, 2025 149 2

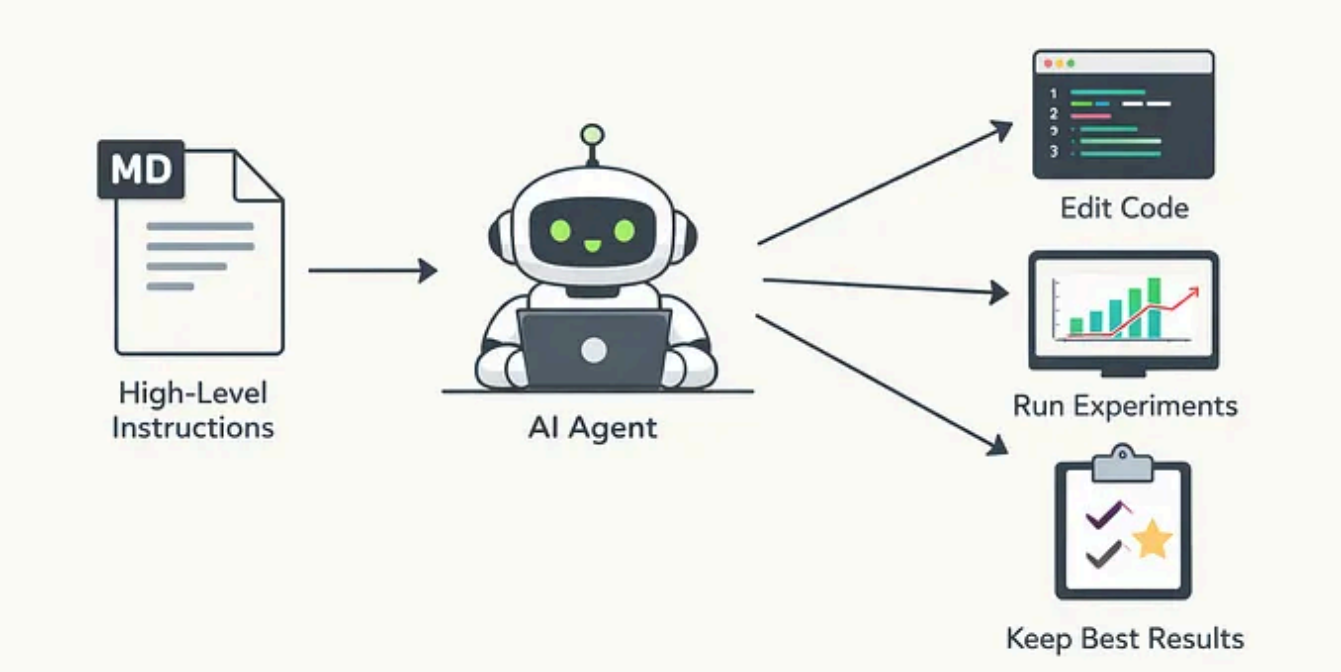


In Neural Notions by Nikhil

How to Set Up Openclaw (Previously Clawdbot / Moltbot) — Step by Step guide to setup a personal bot

Turn your own device into a personal AI assistant that lives inside your favorite apps

Jan 25 386 14



In Neural Notions by Nikhil

Getting Started with Andrej Karpathy's "autoresearch" — Full Guide

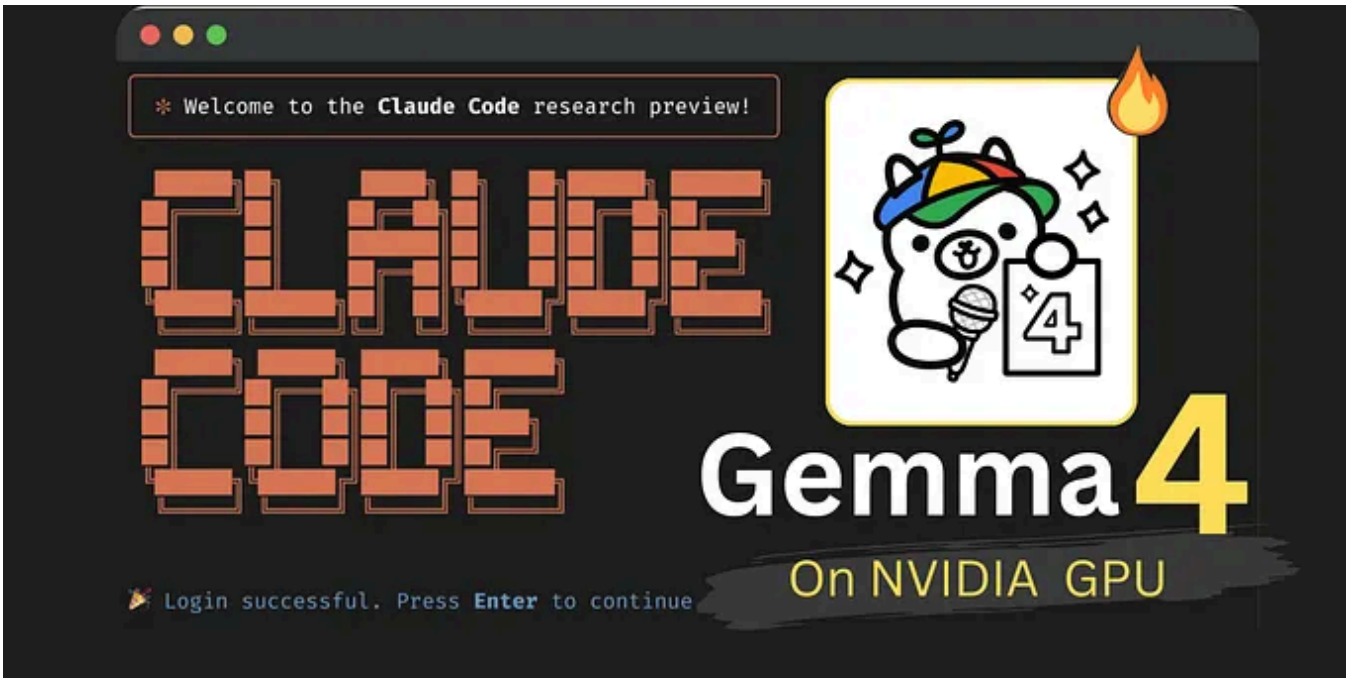
Andrej Karpathy's new project "autoresearch" is a tiny but complete research lab that runs on a single GPU and an AI coding agent.

Mar 8 125 2

See all from Nikhil

See all from Neural Notions

Recommended from Medium



Joe Njenga

I Tried Gemma 4 On Claude Code (And Found New FREE Google Coding Beast)

Imagine running Claude Code with Google's Gemma 4 coding model on an Nvidia GPU — does it get any better?

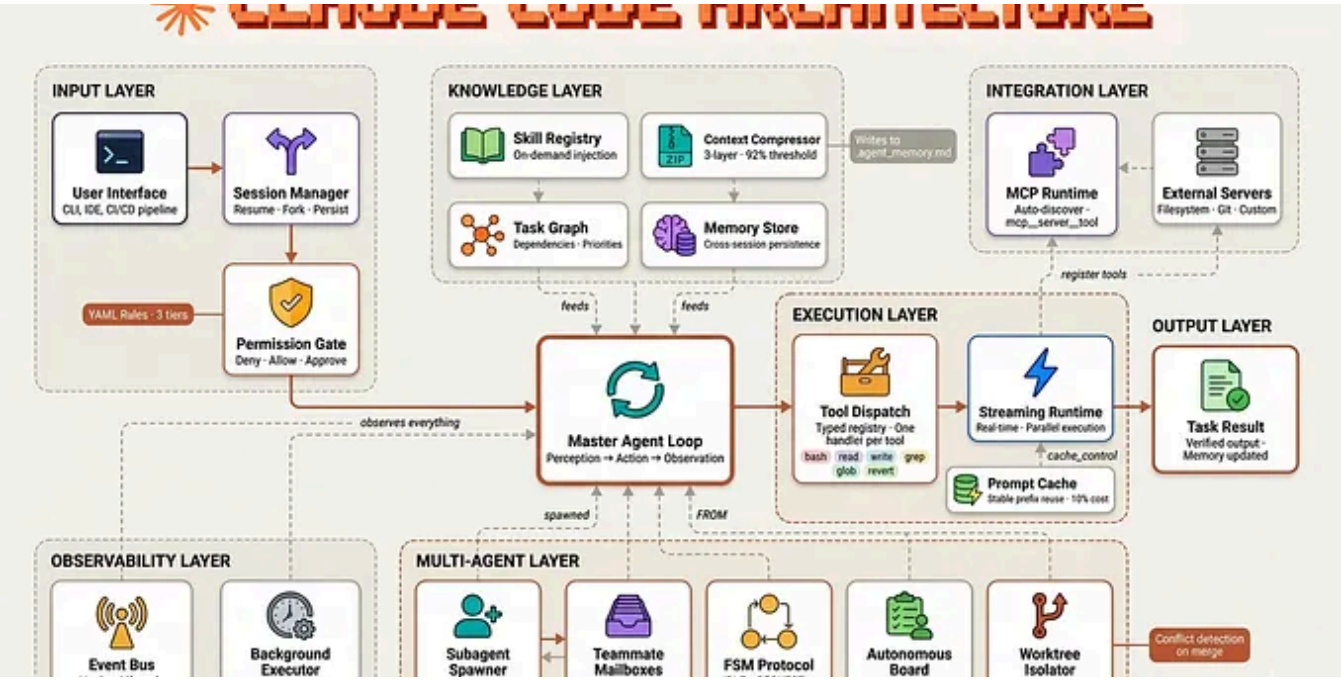
Apr 7 601 21

Balu Kosuri

I used Karpathy's LLM Wiki to build a knowledge base that maintains itself with AI

Here is a scenario most knowledge workers know too well.

Apr 7 742 12



In Level Up Coding by Fareed Khan

Building Claude Code with Harness Engineering

Multi-agents, MCP, skills system, context pipelines and more

Apr 6 1.2K 9

In The Ai Studio by Ai studio

11 Insane Use Cases of OpenClaw AI

What Happens When You Give an AI Agent Access to Your Entire Digital Life

Mar 28 542 17

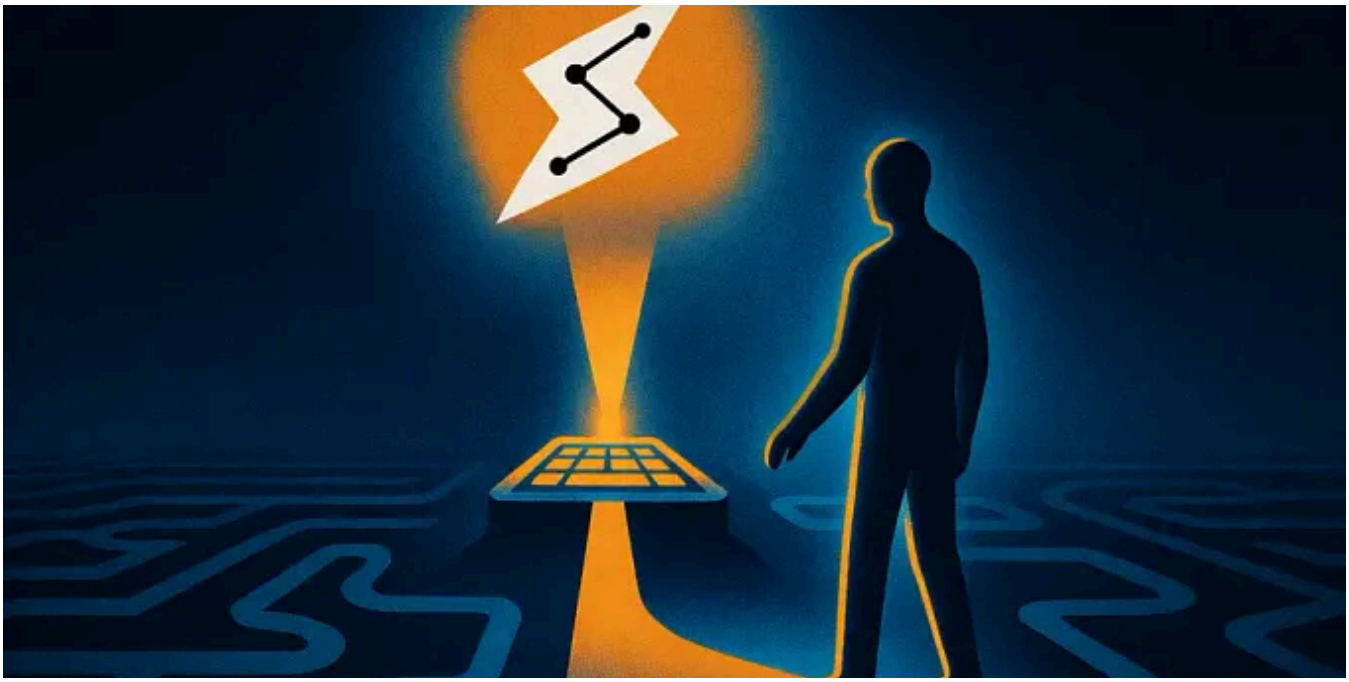
E2B	2.3B	4GB	85	6.1/10	6.5/10	On-device, edge
E4B	4.5B	8GB	38	7.4/10	7.8/10	Local dev, laptops
26B MoE	**3.8B active**	**18GB**	**42**	**9.1/10**	**8.9/10**	**Most use cases**

In Towards AI by Chew Loong Nian

I Tested All 4 Gemma 4 Models: The 26B One Is Cheating (In the Best Way)

Google dropped Gemma 4 this week under Apache 2.0 — free to use, free to commercialize, no strings attached. Four model sizes landed at...

Apr 4 211 2



In Artificial Corner by The PyCoach

Goodbye ChatGPT. Hello Claude Cowork!

Claude Cowork can turn hours of work into minutes (ChatGPT can't do this)

Mar 31 873 25

See more recommendations